

Working with Lambda as a method Parameter :

A method can accept Functional Interface as a method parameter.

We can assign a functional interface OR we can directly pass Lambda as a method arguments as shown in the program below.

```
package com.ravi.lambda;

@FunctionalInterface
interface Printable
{
    void print();
}

public class FunctionalInterfaceAsAParameter {

    public static void main(String[] args)
    {
        //Printable p = () -> IO.println("Printing through Lambda");
        //getPrintable(p);

        getPrintable(()->IO.println("Passing Lambda directly"));
    }

    public static void getPrintable(Printable p1)
    {
        p1.print();
    }
}
```

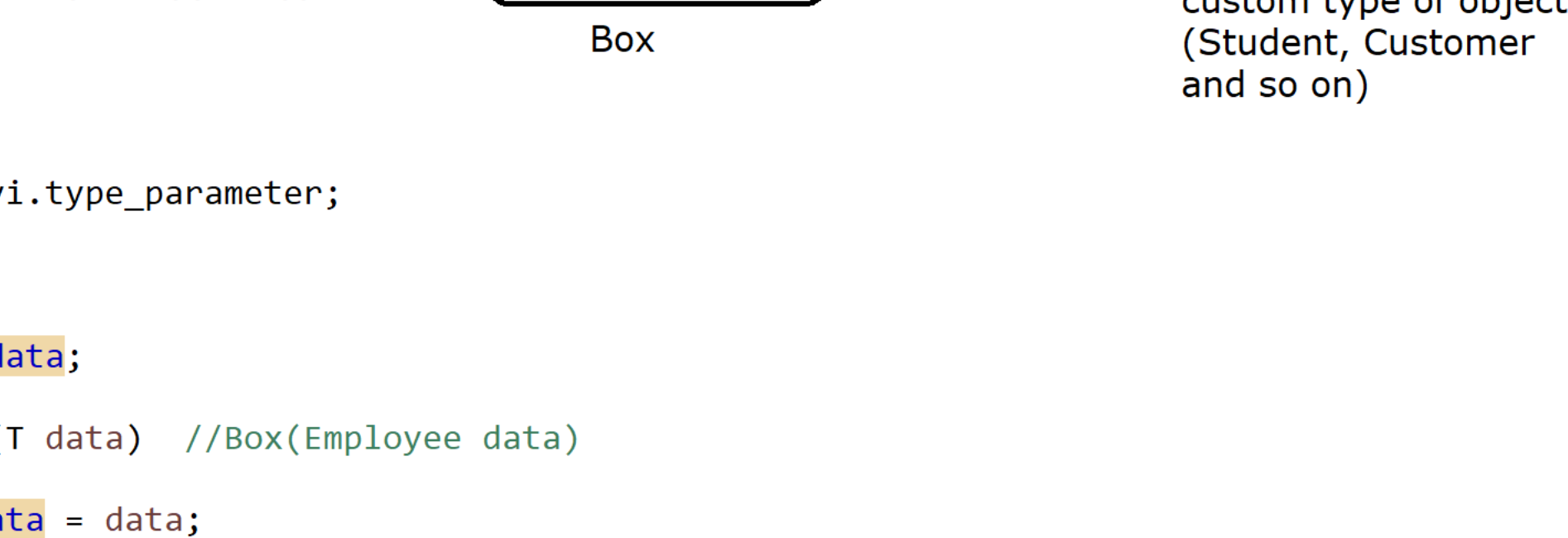
Assigning Lambda to functional interface

What is Type Parameter <T> in Java : [Can work with any type except primitive]

* It is a very special concept in java which is represented by <T> i.e. Type parameter.

* It was introduced from JDK 1.5V to support generic programming.

* By using this type parameter we can make our **java application independent of data type that means it can work with any type of data except primitive. [Primitive types are not allowed]**



```
package com.ravi.type_parameter;

class Box<T>
{
    private T data;

    public Box(T data) //Box(Employee data)
    {
        this.data = data;
    }

    public T getData()
    {
        return this.data;
    }
}

public class TypeParameterDemo
{
    public static void main(String[] args)
    {
        Box<Integer> intType = new Box<Integer>(12);
        IO.println("Integer type is :"+intType.getData());

        Box<Double> doubleType = new Box<Double>(90.67);
        IO.println("Double type is :"+doubleType.getData());

        Box<Character> charType = new Box<Character>('A');
        IO.println("Character type is :"+charType.getData());

        Box<Employee> empType = new Box<Employee>(new Employee(111, "Scott"));
        IO.println("Employee type is :"+empType.getData());
    }
}

record Employee(Integer id, String name)
{
}
}
```

```
package com.ravi.type_parameter;
```

```
@FunctionalInterface
interface Predictable
{
    boolean verify(Integer x);
}
```

```
public class LambdaExample
{
    public static void main(String[] args)
    {
        //Verify whether a number is even OR odd

        Predictable p = (num) -> num % 2==0;

        int number = Integer.parseInt(IO.readLine("Enter a number :"));

        boolean isEven = p.verify(number);

        if(isEven)
        {
            IO.println(number+" is even");
        }
        else
        {
            IO.println(number+" is odd");
        }
    }
}
```

```
}
```

Working with Predefined Functional interfaces :

* In order to work with day to day programming, Java software people has provided following predefined functional interfaces.

- 1) Predicate<T>
- 2) Consumer<T>
- 3) Function<T,R>
- 4) Supplier<T>
- 5) BiPredicate<T,U>
- 6) BiConsumer<T,U>
- 7) BiFunction<T,U,R>
- 8) UnaryOperator<T>
- 9) BinaryOperator<T>

Note : All these predefined functional interfaces are available in **java.util.function** sub package.

1) Predicate<T>

* It is a predefined functional interface available in java.util.function sub package.

```
@FunctionalInterface
public interface Predicate<T>
{
    boolean test(T x);
}
```

* It is used to test **one argument boolean expression**.

* It accepts one abstract method test() which accept single T as a parameter.

```
//Program
-----
package com.ravi.predicate;

import java.util.function.Predicate;

public class PredicateDemo1
{
    public static void main(String[] args)
    {
        // Verify whether a person is eligible for vote or not
        Predicate<Integer> p1 = age -> age >=18;

        int userAge = Integer.parseInt(IO.readLine("Enter your Age :"));

        IO.println("Are you eligible for vote :"+p1.test(userAge));
    }
}
```

```
package com.ravi.predicate;

import java.util.function.Predicate;

public class PredicateDemo2 {

    public static void main(String[] args)
    {
        //Verify whether my name starts with 'S' or nor

        Predicate<String> p2 = name -> name.charAt(0)=='S';
        String userName = IO.readLine("Enter your Name : ").toUpperCase();
        IO.println("Your name starts with 'S' "+p2.test(userName));
    }
}
```

```
package com.ravi.predicate;

import java.util.function.Predicate;

record Student(Integer id, String name, Double marks)
{
}
```

```
public class PredicateDemo3
{
    public static void main(String[] args)
    {
        //Verify whether a student is pass or fail

        Predicate<Student> p3 = student -> student.marks()>=45;
        Student s1 = new Student(111, "Scott", 70D);
        IO.println(s1.name()+" is passed the exam :"+p3.test(s1));
    }
}
```

```
}
```